SMK_CASHEW Full Stack Notes and Interview Guide

Backend + Frontend flow, packages, middleware, modules, debugging lessons, and interview preparation

How to use this PDF
Read Sections 1-7 to understand the project flow. Revise Sections 8-10 before interviews. The examples and bugs are based on the SMK_CASHEW app you built.

1. Project Overview

SMK_CASHEW is a MERN-style ecommerce app for cashew products. The backend is an Express REST API with MongoDB Atlas and Mongoose. The frontend is a Vite React app that uses Axios, React Router, AuthContext, and CartContext.

Layer	Responsibility	Files
Server	Starts Express, connects MongoDB, mounts APIs	backend/src/server.js
Models	Define data shape and validation	User, Product, Cart, Order
Controllers	Business logic and JSON response	auth, product, cart, checkout, admin order controllers
Routes	URL + method mapping	authRoutes, productRoutes, cartRoutes, checkoutRoutes, adminRoutes
Middleware	Reusable logic before controller	protect, adminOnly, validate, notFound, errorHandler
Frontend	Pages, contexts, API calls, UI state	frontend/src

2. End-to-End Flow

1. React page/component triggers an action.
2. Axios sends an API request to the backend base URL.
3. AuthContext adds Authorization: Bearer when logged in.
4. Express server receives the request.
5. Route module matches the path and HTTP method.
6. Middleware runs before controller: auth, validation, admin checks.
7. Controller calls Mongoose model methods.
8. MongoDB returns data.
9. Controller sends JSON response.
10. React stores response in state and re-renders UI.

```
React -> Axios -> Express route -> Middleware -> Controller -> Mongoose Model -> MongoDB -> JSON -> React state
```

3. Backend Architecture

server.js

server.js is the backend entry point. It loads .env, creates the app, connects DB, installs middleware, mounts routes, then starts the server.

```
app.use('/api/products', productRoutes)
app.use('/api/auth', authRoutes)
app.use('/api/cart', cartRoutes)
app.use('/api/checkout', checkoutRoutes)
app.use('/api/admin', adminRoutes)

app.use(notFound)
app.use(errorHandler)
```

Route order matters
All real routes must be mounted before notFound and errorHandler. If adminRoutes is mounted after notFound, /api/admin/orders will never run.

Models, schemas, and mongoose.model

A schema is the blueprint. A model is the database interface. That is why Product.js exports mongoose.model('Product', productSchema), not only productSchema.

Term	Meaning	Example
Schema	Field types, validation, defaults	productSchema
Model	Class with DB methods	Product.find(), Product.create()
Document	One saved record	product._id, product.save()

Controllers

Controller	Important methods	Purpose
authController	register, login, getMe	User creation, login, JWT, current user
productController	getProducts, getProductById, createProduct, updateProduct, deleteProduct	Product listing and admin product management
cartController	getCart, addToCart, updateCartItem, removeCartItem	Cart lifecycle for logged-in users
checkoutController	createOrder, getMyOrders	Convert cart into order and show user orders
adminOrderController	getAllOrders, updateOrderStatus	Admin order dashboard and status updates

4. Authentication and Authorization

Authentication means proving who the user is. Authorization means deciding whether that user can perform an action.

Part	What it does
JWT	Stores signed userId after login/register
protect	Reads Bearer token, verifies it, loads user, assigns req.user
adminOnly	Allows only req.user.role === 'admin'
AuthContext	Stores token, calls /auth/me, sets Axios Authorization header

```
Authorization: Bearer
protect -> jwt.verify -> User.findById(decoded.userId) -> req.user
adminOnly -> req.user.role === 'admin'
```

Debug tip
If admin routes return 403, call GET /api/auth/me with the same token. The token may belong to a customer user even if another user in MongoDB is admin.

5. Product, Cart, Checkout, and Admin Order Flow

Product flow

- GET /api/products returns active products with pagination/filter query support.
- GET /api/products/:id uses MongoDB _id and Product.findById.
- POST/PUT/DELETE /api/products are admin-only.
- DELETE is a soft delete: it sets isActive: false instead of removing the record.

Cart flow

- POST /api/cart accepts productId and quantity.
- Cart stores product ObjectId, quantity, and priceSnapshot.
- GET /api/cart populates items.product to show product details.
- Using populate('item.product') fails because the schema field is items, plural.

Checkout flow

1. Find current user's cart and populate items.product.
2. Reject empty cart.
3. Verify each product is active and stock is enough.
4. Create orderItems from cart items.
5. Calculate subtotal, shippingFee, totalAmount.
6. Create Order document.
7. Decrease product stock.
8. Clear cart.items.
9. Return populated order.

Why cart is empty after checkout
The cart is intentionally cleared because the items moved into an order. Use /api/checkout/my-orders or /api/admin/orders to view order details after checkout.

6. Frontend Architecture

Module	Purpose
services/api.js	One Axios instance with baseURL
AuthContext.jsx	Token, user, login/logout, /auth/me
CartContext.jsx	Cart items and add/update/remove/fetch functions
App.jsx	React Router path mapping
Navbar.jsx	Navigation based on auth/admin state
Products.jsx	Fetches product list
AdminOrders.jsx	Fetches all orders and updates status

React hooks in this app

Hook/pattern	Use	Example
useState	Store render-driving state	products, status, error
useState(() => value)	Lazy initial value once	localStorage token
setState(value)	When new value is known	setStatus('loading')
setState(prev => next)	When next depends on previous	setItems(prev => ...)
useEffect	External sync/fetch	fetch products, fetch current user
useMemo	Memoize context value	AuthContext/CartContext value

7. Packages and Middleware

Backend package	Why used
express	Server, routes, middleware pipeline
mongoose	Schemas, models, MongoDB queries, populate
dotenv	Loads PORT, MONGO_URI, JWT_SECRET, CLIENT_URL
bcryptjs	Password hashing and compare
jsonwebtoken	JWT creation and verification
cors	Allows frontend origin to call backend
helmet	Security HTTP headers
morgan	Request logging
express-rate-limit	Basic abuse protection
express-validator	Body/query/param validation
nodemon	Auto restart during dev

Frontend package	Why used
vite	Fast dev server and build
react/react-dom	Component UI and DOM rendering
react-router-dom	Client routes like /products and /admin/orders
axios	Reusable API client
lucide-react	Icons
tailwindcss	Utility styling
eslint	Finds hook/import/quality problems

8. API Reference

Method	Endpoint	Access	Purpose
GET	/api/health	Public	Health check
GET	/api/products	Public	List products
GET	/api/products/:id	Public	Get product by _id
POST	/api/products	Admin	Create product
PUT	/api/products/:id	Admin	Update product
DELETE	/api/products/:id	Admin	Soft delete product
POST	/api/auth/register	Public	Register
POST	/api/auth/login	Public	Login
GET	/api/auth/me	User	Current user
GET	/api/cart	User	Cart
POST	/api/cart	User	Add item
PUT	/api/cart/:productId	User	Update quantity
DELETE	/api/cart/:productId	User	Remove item
POST	/api/checkout	User	Create order
GET	/api/checkout/my-orders	User	My orders
GET	/api/admin/orders	Admin	All orders
PUT	/api/admin/orders/:id/status	Admin	Update order status

9. Debugging Lessons From This Project

Symptom	Root cause	Fix
Mongo URI undefined	.env had MONG0_URI with zero	Rename to MONGO_URI
Function not defined	Export name did not match function name	Match createProduct/createProduct
JSON parse error	Single quotes or JS object syntax	Use valid JSON double quotes
/auth/me not found	Route was POST but request was GET	Use router.get('/me')
Populate error	item.product vs items.product	Use populate('items.product')
cart before initialization	Used cart.findOne instead of Cart.findOne	Use model Cart
Admin route 404	adminRoutes after notFound	Mount before notFound
Admin access only	Token belonged to customer	Login as admin and verify /auth/me
Atlas crash	Current IP not whitelisted	Add IP in Network Access
Register page blank	/regiter typo	Use /register

10. Interview Questions and Answers

1. What is Express used for?

Express creates the backend server, routes, middleware chain, and JSON responses.

2. What is Mongoose used for?

Mongoose defines schemas/models and provides database methods such as find, create, findById, update, and populate.

3. Why export mongoose.model instead of schema?

The schema is a blueprint. The model is the object that performs database operations.

4. How is _id generated?

MongoDB automatically creates a unique ObjectId for each document.

5. What is JWT?

A signed token that lets the backend identify the user without storing session state on the server.

6. What does protect middleware do?

It reads the Authorization header, verifies token, loads user, and sets req.user.

7. What does adminOnly do?

It checks req.user.role === 'admin' and returns 403 otherwise.

8. Why use bcrypt?

To store password hashes instead of plain passwords.

9. Why use express-validator?

To reject invalid request data before controller/database logic.

10. Why use priceSnapshot?

To remember the item price at cart/order time even if product price changes later.

11. What is populate?

Mongoose replaces an ObjectId reference with document data from the referenced collection.

12. Why clear cart after checkout?

The cart has been converted into an order, so active cart should be empty.

13. Why can admin token still fail?

The token may point to a different user. Always check /auth/me with the same token.

14. What is React Router?

It maps browser paths to React components without full page reload.

15. Why use Axios?

It centralizes API baseURL and headers and simplifies request/response handling.

16. Why use Context API?

To share auth/cart state across components without prop drilling.

17. Can setState be used in useEffect?

Yes, especially after async fetches. Avoid infinite loops caused by dependencies.

18. What does dependency array do?

It controls when an effect reruns.

19. Why use useState lazy initializer?

To compute/read an initial value only once, such as localStorage token.

20. What status codes matter here?

201 create, 400 validation error, 401 no/invalid auth, 403 forbidden, 404 not found, 500 server error.

21. How do you debug Node crashes?

Read stack trace, identify file/line, inspect imports/exports, reproduce with one route, patch root cause.

22. How do you debug Atlas connection failure?

Check MONGO_URI, Atlas Network Access IP whitelist, username/password, cluster status.

11. Revision Checklist

- Explain full request lifecycle from React to MongoDB and back.
- Explain schema vs model vs document.
- Explain protect vs adminOnly.
- Explain why route order matters in Express.
- Explain cart -> checkout -> order flow.
- Explain useState, useEffect, useMemo, and Context in the frontend.
- Explain common backend status codes.

- Explain how to verify token role with /api/auth/me.
- Explain MongoDB Atlas IP whitelist issue.

Best interview framing
When asked about this project, start with the business flow, then the technical architecture, then one debugging story. That shows both product understanding and engineering maturity.